

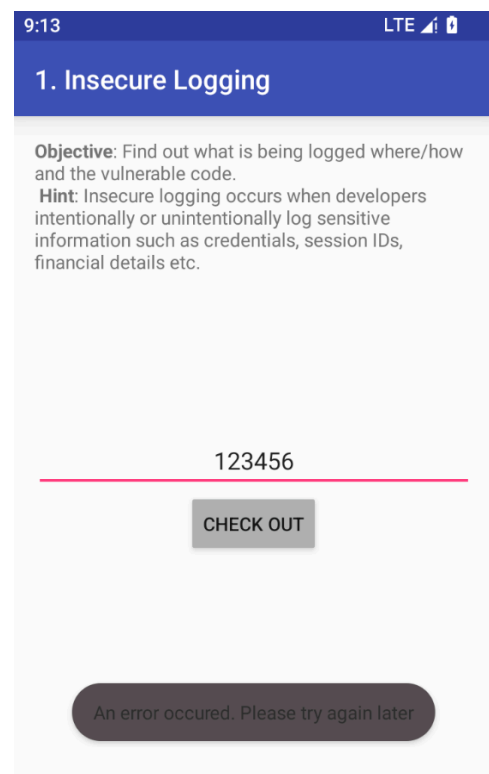
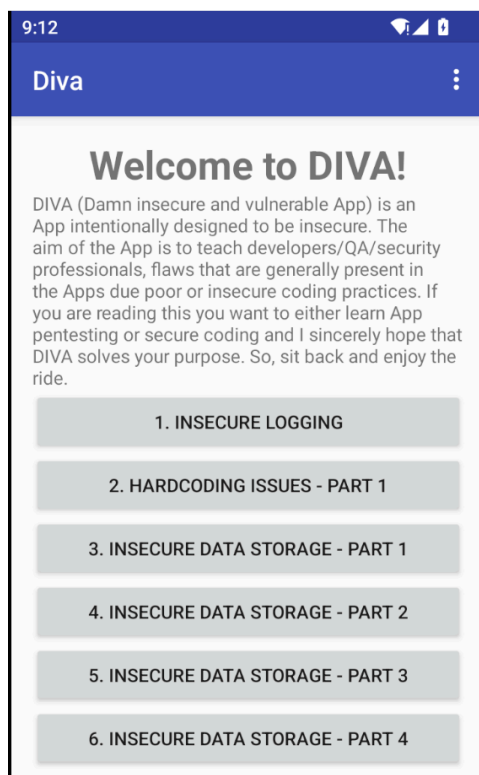
Mobile Forensics Documentation: Analyzing Insecure Logging in the DIVA App

In this task, we will analyze a vulnerable Android application called **DIVA** and identify a common security flaw related to **insecure logging**. Insecure logging occurs when sensitive data (such as credit card information, session IDs, passwords, etc.) is logged in plaintext, which can potentially expose sensitive user data. We will analyze the DIVA application in a **Genymotion emulator** environment using **Kali Linux** as the testing machine.

So setup Genymotion in the Host machine and setup the adb connection using this command,

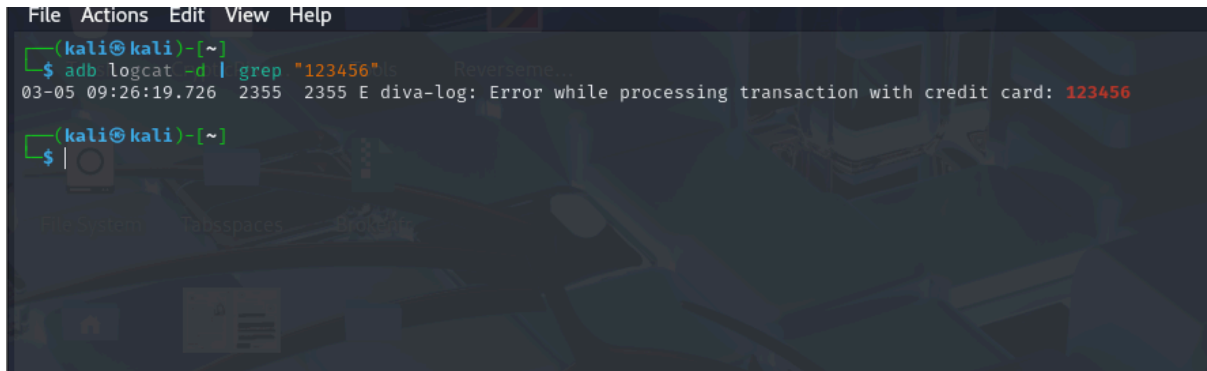
adb connect <ip address>

Let's try the first lab.



So the lab is asking to find out what is being logged where/how and the vulnerable code.

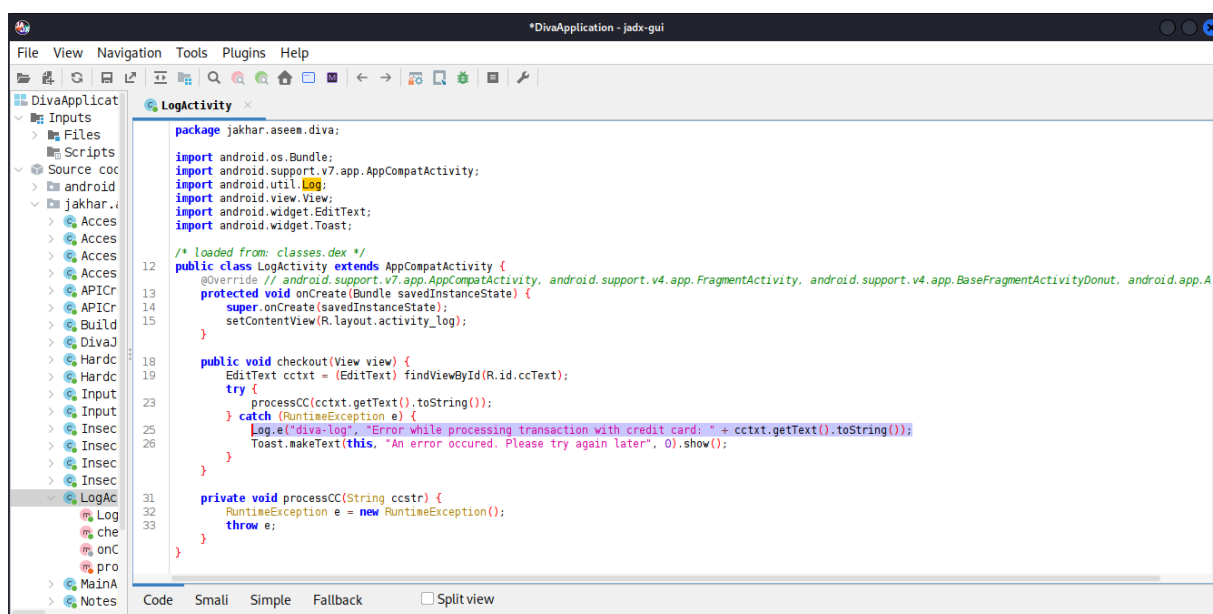
So we will try if this is being logged or not using **adb logcat**



```
File Actions Edit View Help
(kali㉿kali)-[~]
$ adb logcat -d | grep "123456"
03-05 09:26:19.726 2355 2355 E diva-log: Error while processing transaction with credit card: 123456
(kali㉿kali)-[~]
$
```

So when I searched for the credential detail I have entered it was showing on the logcat. So this a big vulnerability which could lead to credential thefts. So now we have to find the vulnerable code which leads to this vulnerability.

I'm using **jadx-gui** to decompile the apk file.



```
*DivaApplication - jadx-gui
File View Navigation Tools Plugins Help
DivaApplication
  Inputs
  Files
  Scripts
  Source code
  android
  jakhar.aseem.diva
    Access
    Access
    Access
    Access
    APICr
    Build
    DivaJ
    Hardc
    Hardc
    Input
    Input
    Insec
    Insec
    Insec
    Insec
    LogActivity
    LogActivity
    onC
    onC
    pro
    MainA
    Notes

package jakhar.aseem.diva;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;

/* loaded from: classes.dex */
public class LogActivity extends AppCompatActivity {
    @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentActivity, android.support.v4.app.BaseFragmentActivity$Donut, android.app.Activity
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_log);
    }

    public void checkout(View view) {
        EditText cctxt = (EditText) findViewById(R.id.ccText);
        try {
            processCC(cctxt.getText().toString());
        } catch (RuntimeException e) {
            Log.e("diva-log", "Error while processing transaction with credit card: " + cctxt.getText().toString());
            Toast.makeText(this, "An error occurred. Please try again later", 0).show();
        }
    }

    private void processCC(String ccstr) {
        RuntimeException e = new RuntimeException();
        throw e;
    }
}

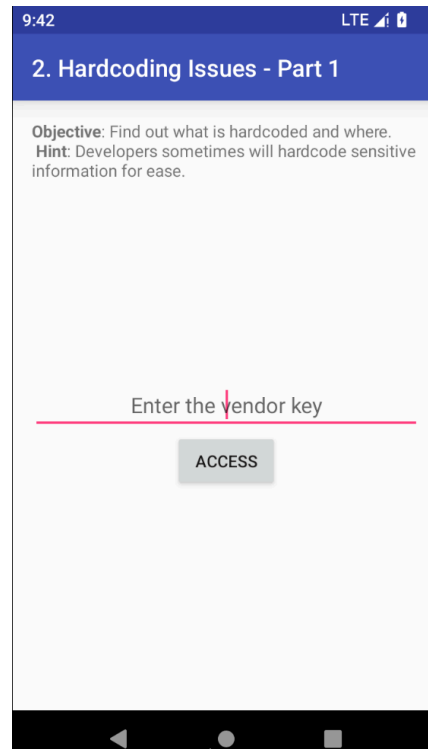
Code Smali Simple Fallback Split view
```

So i found the module, and the selected part in the screenshot is the vulnerability

The provided code is an Android activity (LogActivity) that handles a credit card transaction. The vulnerability lies in the checkout method, specifically in how it logs sensitive information (credit card numbers) and handles exceptions.

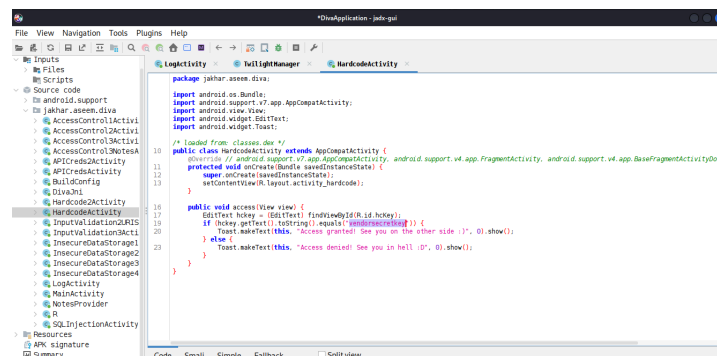
Challenge 2:- HARDCODING ISSUE PART 1

In this task we have to find the hardcoded value, and it is given a hint that, the file is saved in the code. So let's find it from the decompiled code.



Search the complete code of the apk using different keywords like “secret”, “key”, “pass”, “password” etc.

Using the above search you will observe that the hardcoded **activity** class has the vendor key, which you can use to complete this challenge.



9:51

LTE  

2. Hardcoding Issues - Part 1

Objective: Find out what is hardcoded and where.

Hint: Developers sometimes will hardcode sensitive information for ease.

vendorsecretkey

ACCESS

Access granted! See you on the other side :)